EXP NO: 7	GENERATIVE MODELS WITH GANs

AIM:

To build a simple Generative Adversarial Network (GAN) consisting of a Generator and a Discriminator, and train it to generate samples that follow the distribution of real data.

ALGORITHM:

1. Import the required libraries (numpy, matplotlib).
2. Define the real data distribution - a Gaussian with mean 4 and standard deviation 1.25.
3. Build the Generator (noise to sample) and the Discriminator (sample to real/fake probability) as small neural networks trained with the Adam optimiser.
4. Train the Discriminator on real samples (label 1) and generated samples (label 0).
5. Train the Generator to fool the Discriminator by making its outputs be classified as real (label 1).
6. Repeat steps 4 and 5 for 6000 epochs and record the losses.
7. Compare the mean and standard deviation of real and generated data, and plot the distributions.

CODE:

```
# GAN built from scratch with NumPy: learns to generate samples from a 1-D
    Gaussian
# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# 2. Real data distribution: N(mean=4, std=1.25)
REAL_MEAN, REAL_STD = 4.0, 1.25
def real_batch(n):
    return np.random.normal(REAL_MEAN, REAL_STD, (n, 1))
def noise_batch(n):
    return np.random.randn(n, 1)

# 3. A small MLP (1 hidden layer, tanh or ReLU) with Adam optimiser
class MLP:
    def __init__(self, n_in, n_hid, n_out, lr=0.001, act='tanh'):
        self.act = act
        self.p = {'W1': np.random.randn(n_in, n_hid) * 0.5, 'b1':
```

```
            np.zeros(n_hid),
                  'W2': np.random.randn(n_hid, n_out) * 0.5, 'b2':
                      np.zeros(n_out)}
        self.m = {k: np.zeros_like(v) for k, v in self.p.items()}
        self.v = {k: np.zeros_like(v) for k, v in self.p.items()}
        self.t, self.lr = 0, lr
    def forward(self, x):
        self.x = x
        z1 = x @ self.p['W1'] + self.p['b1']
        self.a1 = np.tanh(z1) if self.act == 'tanh' else np.maximum(0, z1)
        return self.a1 @ self.p['W2'] + self.p['b2']
    def backward(self, dout):
        g = {'W2': self.a1.T @ dout, 'b2': dout.sum(0)}
        d_act = (1 - self.a1 ** 2) if self.act == 'tanh' else (self.a1 >
            0).astype(float)
        dz1 = (dout @ self.p['W2'].T) * d_act
        g['W1'] = self.x.T @ dz1
        g['b1'] = dz1.sum(0)
        return dz1 @ self.p['W1'].T, g          # gradient w.r.t. input,
            parameter grads
    def step(self, g, b1=0.5, b2=0.999, eps=1e-8):
        self.t += 1
        for k in self.p:
            self.m[k] = b1 * self.m[k] + (1 - b1) * g[k]
            self.v[k] = b2 * self.v[k] + (1 - b2) * g[k] ** 2
            mh = self.m[k] / (1 - b1 ** self.t)
            vh = self.v[k] / (1 - b2 ** self.t)
            self.p[k] -= self.lr * mh / (np.sqrt(vh) + eps)

sigmoid = lambda z: 1 / (1 + np.exp(-z))
def bce(p, label):
    return -np.mean(label * np.log(p + 1e-8) + (1 - label) * np.log(1 - p +
        1e-8))

# 4. Build Generator (noise -> sample) and Discriminator (sample -> real/fake)
G = MLP(1, 64, 1, lr=0.0003, act='relu')   # ReLU generator
D = MLP(1, 32, 1, lr=0.0003)

# 5. Adversarial training loop
EPOCHS, BATCH = 6000, 128
d_losses, g_losses, snapshots = [], [], {}
for epoch in range(1, EPOCHS + 1):
    # --- Train Discriminator: real -> 1, fake -> 0
    real = real_batch(BATCH)
    fake = G.forward(noise_batch(BATCH))
    p_real = sigmoid(D.forward(real))
    _, g_r = D.backward((p_real - 1) / BATCH)
    p_fake = sigmoid(D.forward(fake))
    _, g_f = D.backward(p_fake / BATCH)
    D.step({k: g_r[k] + g_f[k] for k in g_r})
    d_loss = bce(p_real, 1) + bce(p_fake, 0)

    # --- Train Generator: fool the discriminator (fake -> 1)
    fake = G.forward(noise_batch(BATCH))
    p_gen = sigmoid(D.forward(fake))
```

```
    dx, _ = D.backward((p_gen - 1) / BATCH)
    _, g_g = G.backward(dx)
    G.step(g_g)
    g_loss = bce(p_gen, 1)

    d_losses.append(d_loss); g_losses.append(g_loss)
    if epoch in (1, 1000, 3000, 6000):
        snapshots[epoch] = G.forward(noise_batch(2000)).ravel()
    if epoch % 1000 == 0 or epoch == 1:
        gen = G.forward(noise_batch(2000))
        print(f"Epoch {epoch:5d} | D loss: {d_loss:.3f} | G loss: {g_loss:.3f}
            | "
              f"Generated mean: {gen.mean():.3f} std: {gen.std():.3f}")

# 6. Final comparison
final = G.forward(noise_batch(5000)).ravel()
print(f"\nReal data      -> mean: {REAL_MEAN:.3f}, std: {REAL_STD:.3f}")
print(f"Generated data -> mean: {final.mean():.3f}, std: {final.std():.3f}")

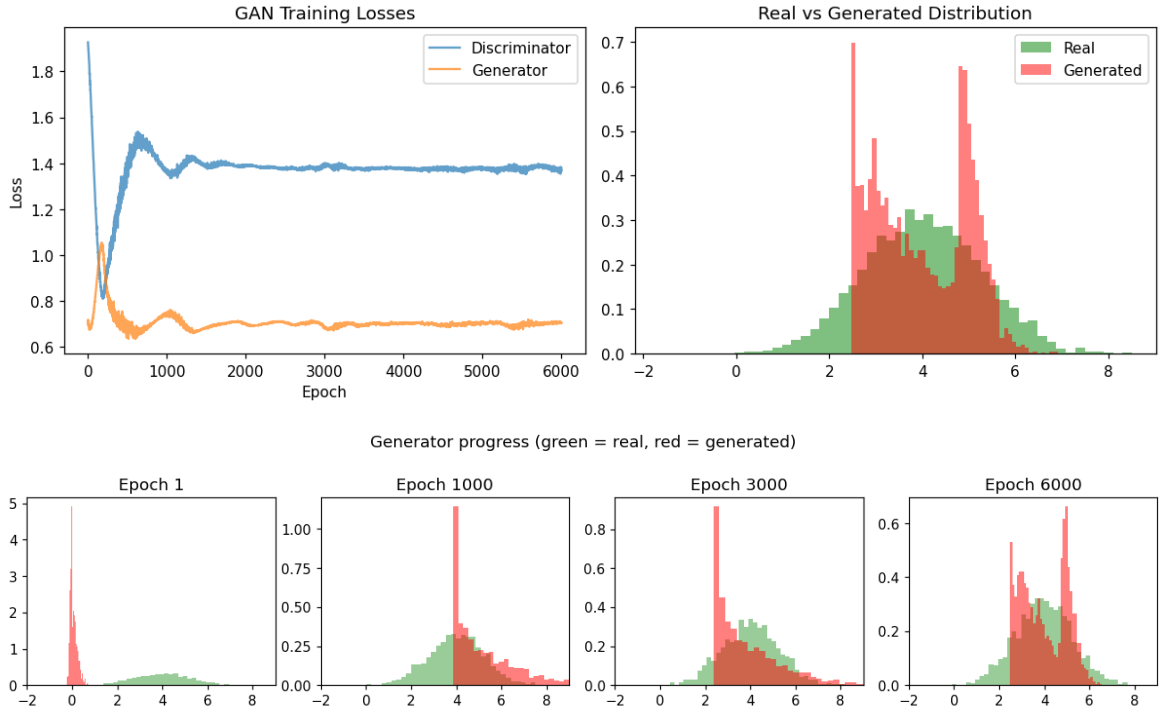
# 7. Plots: loss curves and distribution at different epochs
plt.figure(figsize=(11, 4))
plt.subplot(1, 2, 1)
plt.plot(d_losses, label='Discriminator', alpha=0.7)
plt.plot(g_losses, label='Generator', alpha=0.7)
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("GAN Training Losses");
    plt.legend()
plt.subplot(1, 2, 2)
plt.hist(real_batch(5000).ravel(), bins=50, density=True, alpha=0.5,
    label='Real', color='green')
plt.hist(final, bins=50, density=True, alpha=0.5, label='Generated',
    color='red')
plt.title("Real vs Generated Distribution"); plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 3))
for i, (ep, s) in enumerate(snapshots.items()):
    plt.subplot(1, 4, i + 1)
    plt.hist(real_batch(2000).ravel(), bins=40, density=True, alpha=0.4,
        color='green')
    plt.hist(s, bins=40, density=True, alpha=0.5, color='red')
    plt.title(f"Epoch {ep}"); plt.xlim(-2, 9)
plt.suptitle("Generator progress (green = real, red = generated)")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Epoch     1 | D loss: 1.926 | G loss: 0.710 | Generated mean: 0.048 std: 0.154
Epoch  1000 | D loss: 1.366 | G loss: 0.735 | Generated mean: 5.324 std: 1.534
Epoch  2000 | D loss: 1.389 | G loss: 0.699 | Generated mean: 3.915 std: 0.763
Epoch  3000 | D loss: 1.397 | G loss: 0.682 | Generated mean: 3.879 std: 1.514
Epoch  4000 | D loss: 1.379 | G loss: 0.706 | Generated mean: 4.049 std: 1.120
Epoch  5000 | D loss: 1.371 | G loss: 0.710 | Generated mean: 4.033 std: 1.062
Epoch  6000 | D loss: 1.379 | G loss: 0.705 | Generated mean: 4.030 std: 1.008

Real data      -> mean: 4.000, std: 1.250
Generated data -> mean: 4.005, std: 1.010
```



RESULT:

A simple GAN was successfully implemented and trained. The Discriminator loss settled near 1.38 (close to the equilibrium value of 2ln2), which indicates the Discriminator can no longer easily separate real from generated data. The generated samples had a mean of 4.00 (real: 4.00) and a standard deviation of 1.01 (real: 1.25), so the Generator learned the centre of the distribution well but covered the tails only partly.